

VIT BHOPAL UNIVERSITY

AY 2026-27

CYBERSHIELD

Python-Based Cybersecurity Threat Detection & Incident Analysis Simulator

PROJECT REPORT

Submitted as part of the requirements for B.Tech in Computer Science and Engineering (Cyber Security and Digital Forensics)

Submitted By:

Aastik Tripathi
26BCY10090

1. Introduction

Cybersecurity involves protecting computer systems, users, and data from unauthorized access and other security threats. Security systems continuously deal with different types of events such as successful logins, failed login attempts, suspicious activities, and changes in user privileges.

For beginners, understanding how these events can be collected and analysed can be difficult without a practical example. This project, **CyberShield**, was developed as a Python-based cybersecurity simulation to demonstrate the basic process of security event detection and incident analysis.

The system allows users to generate simulated security events, analyse them for selected threats, classify their risk level, and view incident reports and security statistics. The project uses Python programming concepts such as functions, lists, conditional statements, exception handling, modules, classes, inheritance, encapsulation, and method overriding.

CyberShield is designed as an educational simulation. It does not perform real-world attacks, network monitoring, or intrusion prevention.

2. Problem Statement

Cybersecurity incidents can generate a large number of events, and identifying potentially suspicious activity requires a structured method of analysis. Beginners learning cybersecurity may understand individual concepts but may find it difficult to connect them into a complete working system.

The problem addressed by CyberShield is to create a simple Python-based system that can simulate security events, detect selected threat conditions, classify event risk, and present the results in an understandable form.

The project focuses on demonstrating the basic workflow of **event**

generation → threat detection → risk analysis → incident reporting.

3. Objectives

The main objectives of CyberShield are:

1. To develop a Python-based simulation for generating cybersecurity events.
2. To detect selected security threats using predefined conditions.
3. To classify security events into different risk levels.
4. To generate a basic incident report from recorded events.
5. To provide security statistics for the generated events.
6. To demonstrate Python programming and Object-Oriented Programming concepts in a cybersecurity-related project.
7. To implement input validation and basic error handling.
8. To provide a simple and understandable interface for users learning cybersecurity concepts.

4. Functional Requirements

The functional requirements describe the main operations that CyberShield must perform.

ID	Requirement	Description
FR1	Security Event Generation	The system should allow users to create simulated security events by entering a username, event type, and number of failed attempts.
FR2	Event Viewing	The system should display all security events currently stored during the program session.
FR3	Threat Detection	The system should identify selected threats such as possible brute-force attacks, suspicious login activity, and privilege changes.
FR4	Risk Analysis	The system should classify security events into LOW, MEDIUM, HIGH, or CRITICAL risk levels.
FR5	Incident Reporting	The system should generate a summary containing total events, detected threats, and high/critical-risk events.
FR6	Security Statistics	The system should display statistics such as successful logins, failed logins, detected threats, and risk categories.
FR7	Input Validation	The system should handle invalid numerical input without terminating the application.
FR8	Exit Function	The system should allow the user to safely terminate the application through the Exit option.

5. Non-Functional Requirements

The non-functional requirements describe the quality and operational characteristics of the system.

1. **Usability**
   The system should provide a simple menu-driven interface that can be easily understood by beginners.
2. **Reliability**
   The application should continue running correctly after valid operations and should handle invalid input without crashing.
3. **Maintainability**
   The project should use separate Python modules for event models, threat detection, incident reporting, and statistics to make the code easier to modify.
4. **Error Handling**
   User input should be validated using appropriate exception handling so that invalid values do not terminate the program unexpectedly.
5. **Performance**
   The system should process the stored security events quickly for the intended small-scale simulation.
6. **Security**
   Sensitive event information such as failed-attempt data is protected within the class using encapsulation.
7. **Scalability**
   The modular structure should allow additional threat detection rules and event types to be added in the future.

6. System Architecture And Design

6.1 System Architecture

CyberShield follows a modular architecture in which different responsibilities are divided into separate Python files. The main.py file controls the application flow and user interaction, while the other modules handle specific parts of the system.

The main components are:

- **main.py** — Provides the menu, accepts user input, creates events, and controls the overall program flow.
- **models.py** — Contains the security event classes and their risk-level logic.
- **detectors.py** — Contains the threat detection rules.
- **reports.py** — Generates the incident report.
- **statistics.py** — Calculates and displays security statistics.

This separation makes the project easier to understand, test, and maintain.

Architecture Diagram

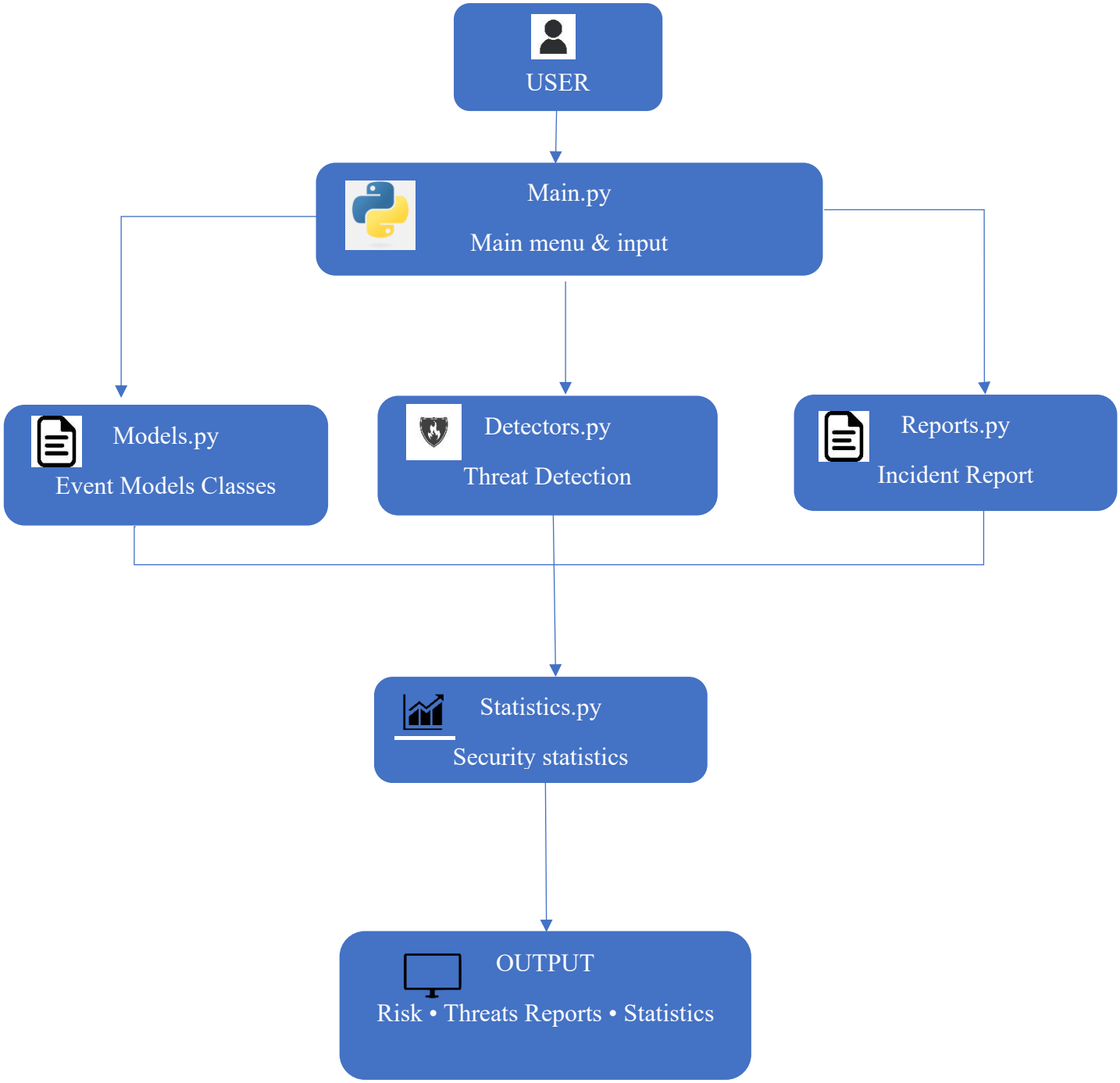


Figure 1: System Architecture Of Cybershield

6.2 System Workflow

The CyberShield workflow begins when the application is started and the main menu is displayed. The user selects an operation, the selected operation is executed, and the result is displayed. The system then returns to the main menu, allowing multiple operations to be performed until the user selects the Exit option.

Work Flow Diagram

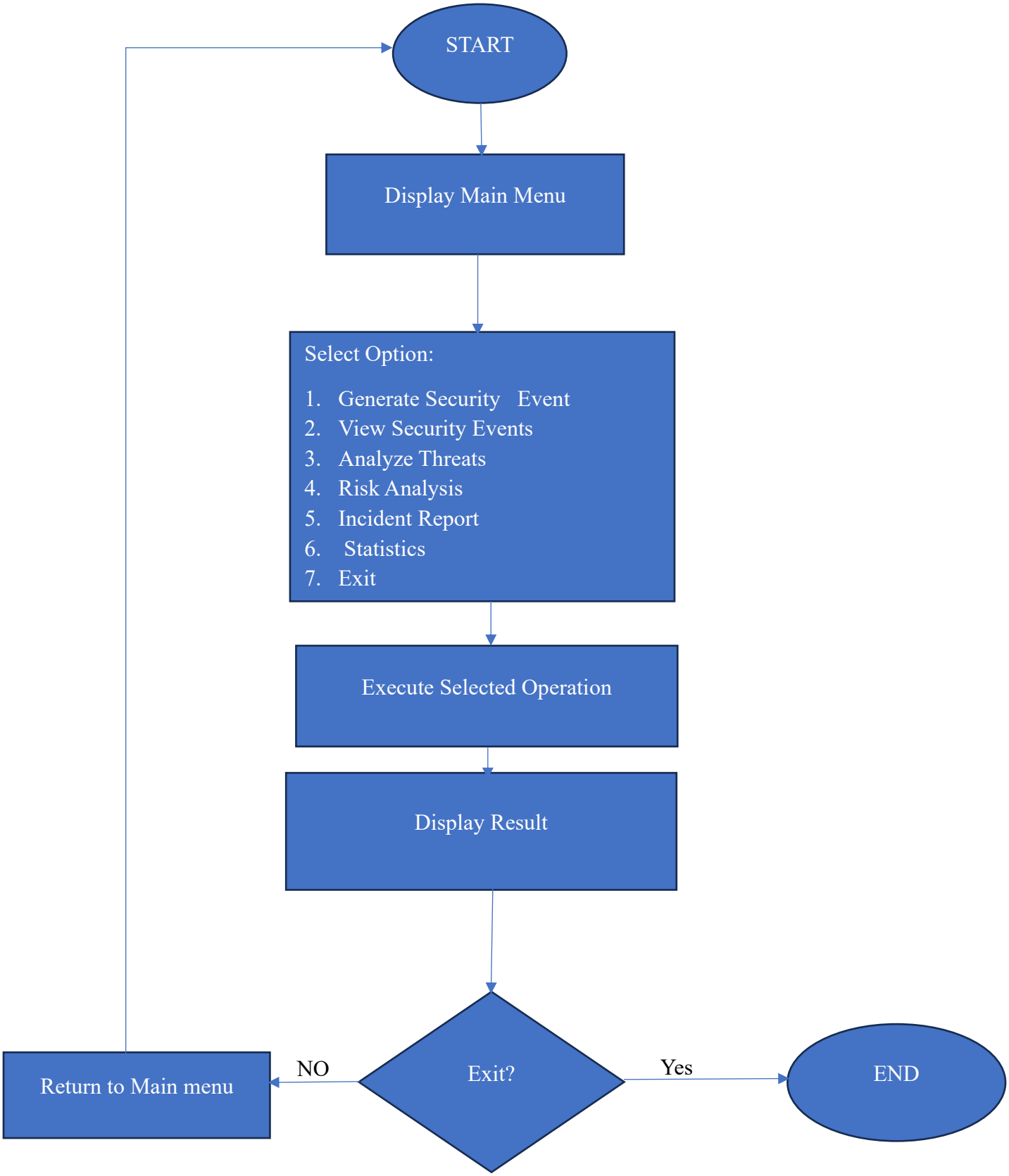
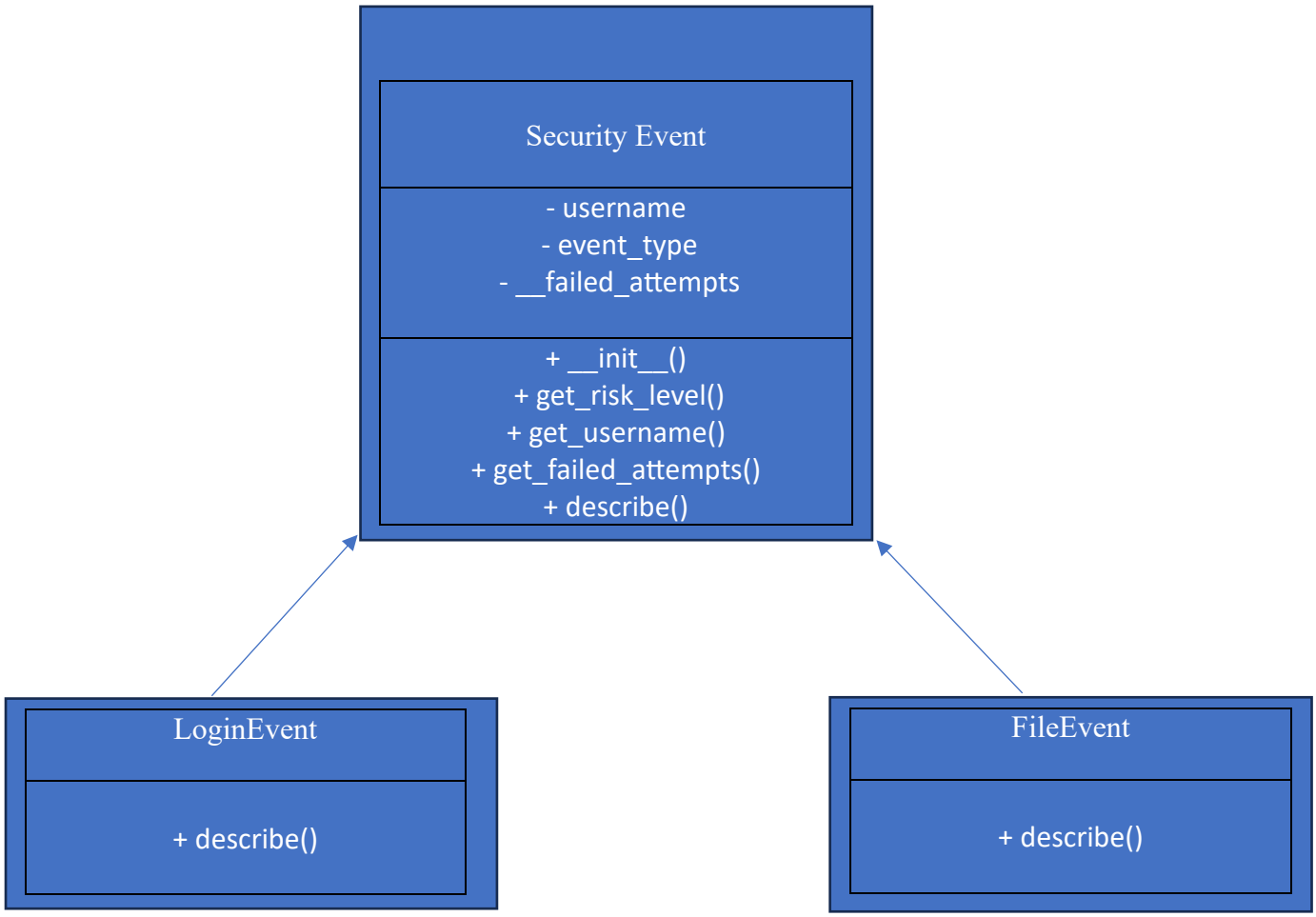


Figure 2: CyberShield System Workflow

6.3 UML / Object-Oriented Design

CyberShield uses Object-Oriented Programming to represent different types of security events. The SecurityEvent class contains common properties and methods, while LoginEvent and FileEvent inherit from it and override the describe() method.

UML Class Diagram



OOP Concepts Demonstrated

- Encapsulation
- Inheritance
- Method Overriding
- Polymorphism
- Classes and Objects

Figure 3: UML Class Diagram of CyberShield

7. PROJECT MODULES AND IMPLEMENTATION

7.1 Project Modules

Module	Purpose
main.py	Controls the main menu, user input, event generation, and application flow
models.py	Contains security event classes and risk-level logic
detectors.py	Contains threat detection methods
reports.py	Generates incident reports
statistics.py	Calculates and displays security statistics
statement.md	Contains the project statement, scope, users, and features
README.md	Contains project documentation and instructions

7.2 Implementation

The main application is controlled by main.py. It provides a menu through which the user can generate events, view events, analyze threats, check risk levels, generate an incident report, and view statistics.

Security events are represented using classes in models.py. The SecurityEvent class stores information such as username, event type, and failed login attempts. The project also uses LoginEvent and FileEvent subclasses to demonstrate inheritance and method overriding.

The ThreatDetector class in detectors.py contains separate methods for detecting possible brute-force attacks, suspicious login activity, and privilege changes.

The reports.py module processes the recorded events and generates an incident summary containing total events, detected threats, and high or critical risk events.

The statistics.py module calculates useful statistics such as successful logins, failed logins, detected threats, and risk categories.

The application also includes input validation using exception handling so that invalid numerical input does not terminate the program.

7.3 Object-Oriented Programming Concepts Used

The project applies several Python Object-Oriented Programming concepts:

- **Classes and Objects** — used to represent security events.
- **Constructor** — initializes event information.
- **Encapsulation** — failed login attempts are stored using a private attribute.
- **Inheritance** — LoginEvent and FileEvent inherit from SecurityEvent.
- **Method Overriding** — subclasses provide their own describe() method.
- **Polymorphism** — different event objects can use the same describe() method while producing different descriptions.

8. THREAT DETECTION AND RISK ANALYSIS

8.1 Threat Detection

CyberShield uses rule-based logic to identify selected security conditions from simulated events. The system does not perform real-world attacks or network monitoring. It only analyzes the events generated within the application.

Threat Condition	**Detection Rule**
Possible Brute Force Attack	Failed attempts ≥ 5
Suspicious Login Activity	Event type is SUSPICIOUS_LOGIN
Privilege Change	Event type is PRIVILEGE_CHANGE

8.2 Risk Classification

Each security event is assigned a risk level according to the number of failed login attempts.

Failed Attempts	Risk Level
0–2	LOW
3–4	MEDIUM
5–7	HIGH
8 or more	CRITICAL

This provides a simple way to classify the severity of simulated security events.

8.3 Incident Analysis

The incident report summarizes the overall security state of the generated events. It displays the total number of events, detected threats, and high or critical risk events. Individual event details are also displayed with their corresponding risk level and status.

The system uses three basic statuses:

- **NORMAL** — no threat detected and low risk.
- **REVIEW** — medium-risk event requiring attention.
- **THREAT DETECTED** — one or more threats identified.

8.4 Statistics

The statistics module provides an overview of the generated security events. It displays the number of successful logins, failed logins, detected threats, low-risk events, and high/critical-risk events.

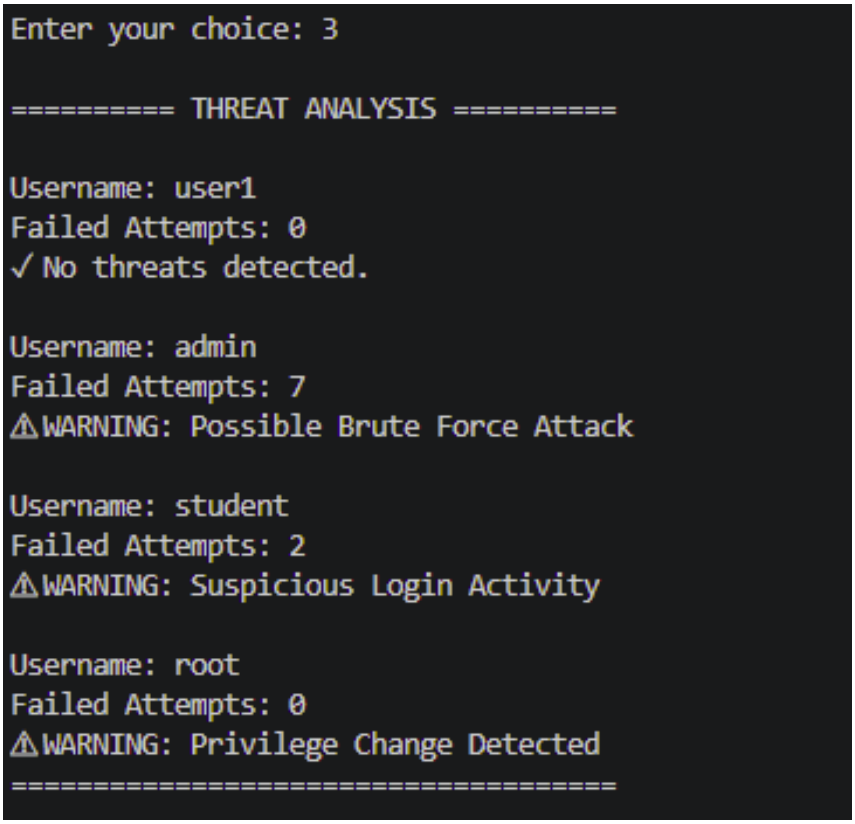
This helps the user understand the overall results of the simulation.

9. SCREENSHOTS AND RESULTS

The completed CyberShield application was tested using different simulated security events. The screenshots below demonstrate the major functions of the system, including threat detection, risk analysis, input validation, and security statistics.

9.1 Threat Analysis

The threat analysis module was tested with different types of security events. The system successfully identified possible brute-force activity, suspicious login activity, and privilege change activity while showing no threat for a normal login event.



9.2 Risk Analysis

The risk analysis module was tested with multiple events having different numbers of failed login attempts. The system correctly classified the events into appropriate risk levels and displayed the detected threats for each event.

```
Enter your choice: 4

========== RISK ANALYSIS ==========

Username: user1
Failed Attempts: 0
Risk Level: LOW
Detected Threats: None

Username: admin
Failed Attempts: 7
Risk Level: HIGH
Detected Threats:
- Possible Brute Force Attack

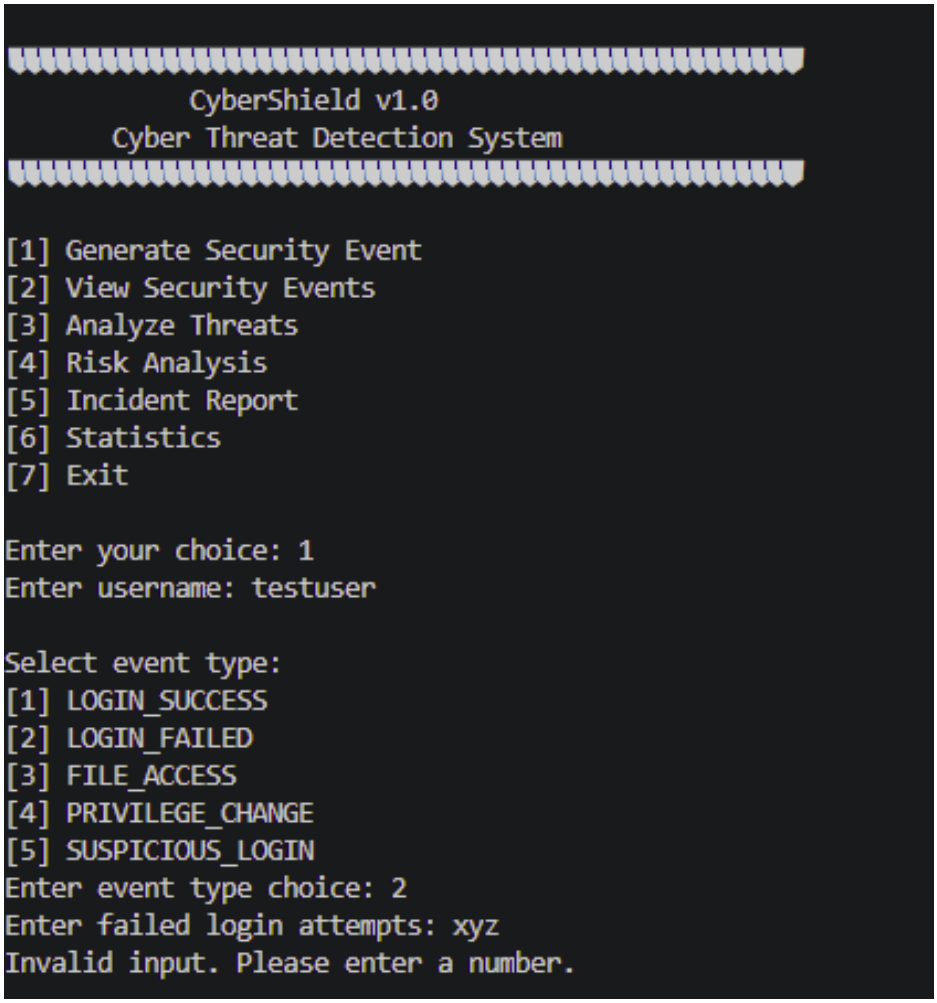
Username: student
Failed Attempts: 2
Risk Level: LOW
Detected Threats:
- Suspicious Login Activity

Username: root
Failed Attempts: 0
Risk Level: LOW
Detected Threats:
- Privilege Change Detected

Username: attacker
Failed Attempts: 10
Risk Level: CRITICAL
Detected Threats:
- Possible Brute Force Attack
==================================
```

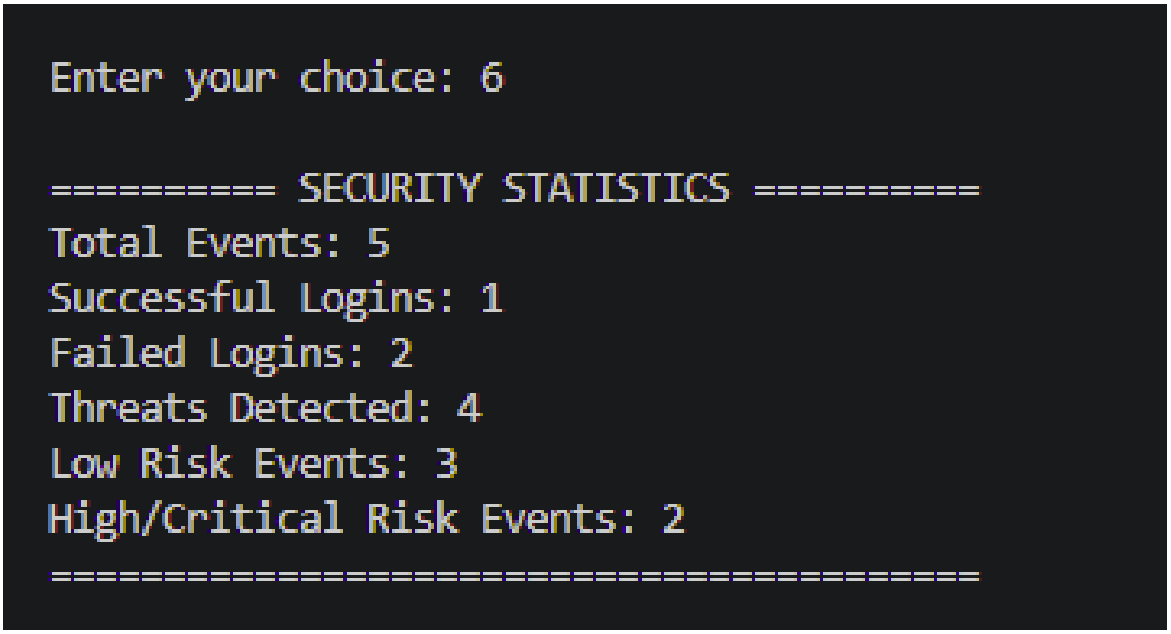
9.3 Input Validation

An invalid value was entered when the system requested the number of failed login attempts. The application displayed an appropriate error message instead of terminating unexpectedly.



9.4 Security Statistics

The statistics module was tested using multiple generated security events. The system successfully displayed the total number of events, successful logins, failed logins, detected threats, low-risk events, and high/critical-risk events.



10. TESTING

The CyberShield application was manually tested using different security event scenarios and user inputs. The purpose of testing was to verify that the system correctly detects threats, assigns risk levels, generates statistics, handles invalid input, and operates correctly when no events are available.

10.1 Test Cases

Test Case	Input / Scenario	Expected Result	Actual Result	Status
TC01	Successful login with 0 failed attempts	LOW risk and no threat	LOW risk and no threat detected	PASS
TC02	LOGIN_FAILED with 7 failed attempts	HIGH risk and brute-force warning	HIGH risk and brute-force warning displayed	PASS
TC03	SUSPICIOUS_LOGIN event	Suspicious login detected	Suspicious Login Activity detected	PASS
TC04	PRIVILEGE_CHANGE event	Privilege change detected	Privilege Change Detected displayed	PASS
TC05	LOGIN_FAILED with 10 failed attempts	CRITICAL risk and brute-force warning	CRITICAL risk and brute-force warning displayed	PASS

Test Case	Input / Scenario	Expected Result	Actual Result	Status
TC06	Invalid failed-attempt input such as xyz	Error message without program crash	Invalid input message displayed and program continued	PASS
TC07	Multiple events in statistics module	Correct event and threat counts	Statistics displayed correctly	PASS
TC08	Options 2–6 with no recorded events	Informative empty-state message	"No security events available." displayed without crash	PASS

10.2 Testing Result

All major functional areas of CyberShield were tested successfully. The application correctly handled normal events, threat conditions, different risk levels, invalid input, statistics generation, and empty event states.

The testing confirmed that the implemented modules work together correctly and that invalid user input does not cause the application to terminate unexpectedly.

11. CHALLENGES AND LEARNING OUTCOMES

11.1 Challenges Faced

During the development of CyberShield, several challenges were encountered while designing and implementing the system.

One of the main challenges was organizing the project into multiple Python modules while keeping the program functions connected correctly. Separating the event models, threat detection, reporting, and statistics required careful handling of imports and data flow.

Another challenge was implementing the threat detection logic in a simple and understandable way. The detection conditions had to produce consistent results for different event types and failed login attempts.

Implementing Object-Oriented Programming concepts was also a learning challenge. Classes, inheritance, encapsulation, and method overriding were used to structure the security event system.

Input validation was another important part of the implementation. The application had to handle invalid user input without terminating unexpectedly.

11.2 Learning Outcomes

This project provided practical experience with Python programming and its application to a cybersecurity-related problem.

Through the development of CyberShield, the following concepts were strengthened:

- Python functions and conditional statements
- Lists and data handling
- Modules and project organization
- Classes and objects
- Constructors
- Encapsulation
- Inheritance
- Method overriding and polymorphism
- Exception handling
- Input validation
- Basic rule-based threat detection
- Software testing and debugging
- Git and GitHub project organization
- Technical documentation and report preparation

The project also helped in understanding how a simple cybersecurity workflow can be represented using programming concepts. Instead of only studying individual Python features, they were combined to build a complete working application.

12. FUTURE ENHANCEMENTS

CyberShield can be extended in several ways in the future:

1. **File-Based Data Storage** — Store security events in files so that data remains available after the program is closed.
2. **Database Integration** — Use a database to manage a larger number of security events.
3. **More Threat Detection Rules** — Add detection for additional suspicious activities and event patterns.
4. **Graphical User Interface** — Develop a GUI to make the application easier to use.
5. **Advanced Reporting** — Generate detailed reports containing additional event and risk information.
6. **Visualization** — Add charts and graphs to represent security statistics visually.
7. **Real-Time Monitoring** — In a future advanced version, the system could process security logs from a controlled environment.
8. **Machine Learning Integration** — Future versions could explore machine-learning techniques for identifying unusual event patterns.

These enhancements would extend CyberShield from a basic educational simulation into a more advanced security-event analysis platform.

13. Conclusion

CyberShield was developed as a Python-based cybersecurity threat detection and incident analysis simulator. The project demonstrates how basic security events can be generated, analyzed, classified, and summarized using Python programming concepts.

The system successfully implements security event generation, threat detection, risk analysis, incident reporting, security statistics, and input validation. Object-Oriented Programming concepts such as classes, inheritance, encapsulation, and polymorphism were also applied to structure the project.

The testing performed on different event scenarios showed that the system correctly identified the implemented threat conditions and assigned appropriate risk levels. The project provided practical experience in Python programming, modular development, debugging, testing, documentation, and applying programming concepts to a cybersecurity-related problem.

Although CyberShield is an educational simulation and does not perform real-world monitoring or attacks, it provides a foundation that can be extended with persistent storage, graphical interfaces, advanced detection rules, visualization, and other security-analysis features.

14. REFERENCES

1. Python Software Foundation, **Python Documentation**, Python 3 Documentation.
2. Python Software Foundation, **Python Tutorial**, Python Documentation.
3. OWASP Foundation, **OWASP Top 10 Web Application Security Risks**.
4. VIT Bhopal University, **Python Programming Project Guidelines**, Academic Year 2026–27.
5. Course lecture materials and practical exercises related to Python programming and Object-Oriented Programming.
6. Vityarthi Course Syllabus